CSE752: Deep Neural Networks

Week 1 Lecture Notes

Foundations: Neurons, MLPs, and Backpropagation

Department of Computer Science and Engineering
School of Data and Sciences, BRAC University

Fall 2026 | Slides: `CSE752_W01_Slides.tex` | https://cse.bracu.ac.bd/course/view/CSE752

How to use these notes

The slide deck is the in-class path. These notes hold the same mathematics with the algebra written out, four fully worked examples, and eight problems with solutions. Read Sections **??**–**??** before Home Practice. Attempt the problems in Section **??** before turning to the solutions.

Suggested timing for a 3-hour meeting. Notation and one logistic neuron (35 min); activations and XOR (30 min); losses and the $2 \to 2 \to 1$ forward pass (30 min); chain rule and backprop on that net (40 min); problems 1, 2, 6, and 7 (35 min); Home Practice briefing (10 min).

Reading. Prince, *Understanding Deep Learning*, Ch. 3–4. Goodfellow, Bengio, Courville, *Deep Learning*, Ch. 6. Optional scalar walkthrough: Nielsen, *Neural Networks and Deep Learning*, Ch. 2.

Convention. One example is a column. Layer ℓ has

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(0)} = x,$$

with $W^{(\ell)}$ shaped (outputs $\times$ inputs). Section **??** translates this into the row-batch layout used by Home Practice.

Contents

1 What a neuron computes

1.1 Logistic regression as a one-neuron net

1 What a neuron computes

Definition 1.1 (Affine unit)**.** For $x \in \mathbb{R}^d$, a unit with weights $w \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$ produces the pre-activation $z = w^\top x + b$ and the activation $a = f(z)$, where $f : \mathbb{R} \to \mathbb{R}$ is applied to the scalar z.

An affine function of an affine function is affine. If two layers use the identity,

$$W^{(2)}\big(W^{(1)}x + b^{(1)}\big) + b^{(2)} = (W^{(2)}W^{(1)})x + (W^{(2)}b^{(1)} + b^{(2)}).$$

No choice of the two matrices represents a nonlinear decision boundary. The activation is the reason a multilayer network is not just a larger linear model.

1.1 Logistic regression as a one-neuron net

Let $y \in \{0, 1\}$ and $\hat{y} = \sigma(z)$ with the logistic sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{1}$$

Binary cross-entropy on one example is

$$L(z, y) = -\Big(y \log \sigma(z) + (1 - y) \log\big(1 - \sigma(z)\big)\Big). \tag{2}$$

Proposition 1.1 (Sigmoid derivative)**.** $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$*, and* $0 < \sigma'(z) \le 1/4$*, with equality only at* $z = 0$*.*

Proof. Differentiate $\sigma(z) = (1 + e^{-z})^{-1}$:

$$\sigma'(z) = (1 + e^{-z})^{-2} \cdot e^{-z} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\big(1 - \sigma(z)\big).$$

Let $s = \sigma(z) \in (0, 1)$. The function $s(1 - s)$ is a downward parabola on $[0, 1]$ with vertex at $s = 1/2$, where its value is $1/4$. And $s = 1/2$ only at $z = 0$. □

Proposition 1.2 (BCE through a sigmoid)**.** *If L is* (**??**)*, then* $\partial L/\partial z = \sigma(z) - y$.

Proof. Set $\hat{y} = \sigma(z)$. Differentiate in the probability first:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}.$$

Multiply by Proposition **??**:

$$\frac{\partial L}{\partial z} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y.$$

□

If instead $L = \frac{1}{2}(\hat{y}-y)^2$, the chain stops one step earlier and

$$\frac{\partial L}{\partial z} = (\hat{y}-y)\,\sigma'(z).$$

On a confidently wrong saturated prediction, $\hat{y} \approx 0$ or 1, one has $\sigma' \approx 0$, so the parameter update is tiny. Binary cross-entropy cancels that factor. This is the reason the binary head is sigmoid-plus-BCE, not sigmoid-plus-squared-error.

Example 1.1 (One step of logistic regression)**.** Take $w = 2$, $b = -2$, $x = 1$, $y = 1$.

$$z = 2 \cdot 1 - 2 = 0, \qquad \hat{y} = \sigma(0) = \frac{1}{2}, \qquad L = \ln 2.$$

By Proposition **??**,

$$\frac{\partial L}{\partial z} = -\frac{1}{2}, \qquad \frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = -\frac{1}{2}, \qquad \frac{\partial L}{\partial b} = -\frac{1}{2}.$$

The gradient step $w \leftarrow w - \eta \partial L/\partial w$ increases w. The score $z = wx + b$ on this positive x increases, and $\hat{y}$ moves toward 1.

1.2 A numerically stable form of (??)

For $z \geq 0$,

$$\log \sigma(z) = -\log(1 + e^{-z}),$$
$$\log\big(1 - \sigma(z)\big) = -z - \log(1 + e^{-z}),$$

and substitution into (**??**) produces

$$L = (1-y)z + \log(1 + e^{-z}).$$

The case $z < 0$ is symmetric after writing the exponential of $|z|$. Both cases are covered by

$$L(z, y) = \max(z, 0) - yz + \log\big(1 + e^{-|z|}\big). \tag{3}$$

Equation (**??**) is what a careful implementation evaluates. Clipping $\hat{y}$ away from $\{0, 1\}$ before calling log, as in Home Practice, is a simpler approximation of the same idea.

2 Activation functions

Name	$f(z)$	$f'(z)$	Range
Sigmoid	$\sigma(z)$ in (**??**)	$\sigma(z)(1-\sigma(z))$	$(0, 1)$
tanh	$\tanh z = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2 z$	$(-1, 1)$
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$	$[0, \infty)$
Leaky ReLU	$\max(\alpha z, z)$, $\alpha = 0.01$	α if $z < 0$, else 1	$\mathbb{R}$

At $z = 0$ the ReLU derivative is a subgradient, any number in $[0, 1]$. Implementations use 0. The practical consequence is one-sided: if $z < 0$, this unit contributes nothing upstream. Its incoming weights, its bias, and (because the activation value is also 0) its outgoing weight all receive zero gradient from this example. Across a batch the unit can still learn from other rows. If every row kills it, the unit is frozen for the rest of training. That is a dead ReLU.

Leaky ReLU replaces the zero derivative by α, so a dead-looking unit still receives a small update.

Universal approximation, stated only. A single hidden layer of sigmoids, with enough units, can approximate any continuous function on a compact subset of $\mathbb{R}^d$ to any uniform tolerance (Cybenko, 1989; Hornik, 1991). The theorem does not produce the width, the weights, or a training procedure. Week 1 uses it as a warning against two opposite mistakes: "a hidden layer cannot represent XOR" (false, and disproved by the next example) and "a wide sigmoid net will be easy to optimize" (not implied).

Example 2.1 (XOR by hand)**.** The four points $(0, 0)$, $(1, 0)$, $(0, 1)$, $(1, 1)$ with labels $0, 1, 1, 0$ are not linearly separable. If $w_1 x_1 + w_2 x_2 + b$ were positive on the two class-1 points and negative on the two class-0 points, then

$$\begin{aligned} b &< 0, \\ w_1 + b &> 0, \\ w_2 + b &> 0, \\ w_1 + w_2 + b &< 0. \end{aligned}$$

The middle pair says $w_1 > -b$ and $w_2 > -b$. Combined with $b < 0$,

$$w_1 + w_2 + b > (-b) + (-b) + b = -b > 0,$$

which contradicts the last inequality.

The following ReLU network fits the four points exactly. Let

$$z^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \qquad a = \mathrm{ReLU}(z^{(1)}), \qquad \hat{y} = a_1 - 2a_2.$$

x_1	x_2	z_1	z_2	a_1	a_2	$\hat{y}$
0	0	0	−1	0	0	0
1	0	1	0	1	0	1
0	1	1	0	1	0	1
1	1	2	1	2	1	0

The second unit is an AND detector. Subtracting twice its output from the OR-like first unit leaves XOR. Nine scalar parameters are present if a bias is counted on the output; the output bias here is zero.

3 Multilayer perceptrons

Definition 3.1 (MLP)**.** Fix dimensions $n_0 = d$ and $n_1, \ldots, n_L$. Set $a^{(0)} = x \in \mathbb{R}^d$. For $\ell = 1, \ldots, L$,

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}},\ b^{(\ell)} \in \mathbb{R}^{n_\ell}, \tag{4}$$

$$a^{(\ell)} = f^{(\ell)}(z^{(\ell)}), \tag{5}$$

with $f^{(\ell)}$ applied independently to each coordinate. The network prediction is $a^{(L)}$. Hidden maps $f^{(1)}, \ldots, f^{(L-1)}$ in this course are ReLU unless a problem says otherwise.

A map $\mathbb{R}^{n_{\text{in}}} \to \mathbb{R}^{n_{\text{out}}}$ contributes $n_{\text{out}}n_{\text{in}} + n_{\text{out}}$ parameters. For $784 \to 128 \to 64 \to 10$,

$$784 \cdot 128 + 128 = 100{,}480,$$
$$128 \cdot 64 + 64 = 8{,}256,$$
$$64 \cdot 10 + 10 = 650,$$

and the total is 109,386. The first matrix dominates because the input dimension dominates.

4 Losses

4.1 Squared error

For a scalar prediction, $L = \frac{1}{2}(\hat{y} - y)^2$ has derivative $\hat{y} - y$ in $\hat{y}$. The factor $\frac{1}{2}$ makes that derivative clean. It does not change the location of the minimum. For a vector output the same construction is $\frac{1}{2}\|\hat{y} - y\|_2^2$, and the gradient in $\hat{y}$ is $\hat{y} - y$.

4.2 Softmax cross-entropy

Definition 4.1 (Softmax)**.** For logits $z \in \mathbb{R}^K$,

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}. \tag{6}$$

Subtracting a constant m from every logit multiplies numerator and denominator by e^{-m} and does not change p. Choosing $m = \max_j z_j$ keeps the exponentials in $(0, 1]$.

Let $y \in \mathbb{R}^K$ be one-hot, so $y_k \in \{0, 1\}$ and $\sum_k y_k = 1$. The cross-entropy is

$$L(z, y) = -\sum_{k=1}^{K} y_k \log p_k = -\log p_{k^\star} \tag{7}$$

when $k^\star$ is the true class.

Proposition 4.1 (Softmax cross-entropy gradient)**.** *If $p = \text{softmax}(z)$ and L is (??), then $\nabla_z L = p - y$.*

Proof. Write $S = \sum_j e^{z_j}$, so $p_i = e^{z_i}/S$. For the Jacobian of softmax,

$$\frac{\partial p_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i} S - e^{z_i} e^{z_j}}{S^2} = p_i(\delta_{ij} - p_j).$$

Then

$$\frac{\partial L}{\partial z_j} = -\sum_{k=1}^{K} y_k \cdot \frac{1}{p_k} \cdot \frac{\partial p_k}{\partial z_j} = -\sum_{k=1}^{K} y_k(\delta_{kj} - p_j) = -y_j + p_j \sum_{k=1}^{K} y_k = p_j - y_j,$$

where the last step uses $\sum_k y_k = 1$. □

Two checks follow at once. Probabilities sum to 1 by construction of (??). The residual $p - y$ sums to 0 because both p and y sum to 1. If a coded gradient fails either check, the bug is upstream of the parameter update.

Example 4.1 (Softmax with exact fractions)**.** Let $z = (\ln 2,\, 0,\, 0)$. Then $e^z = (2, 1, 1)$, $S = 4$, and $p = (1/2,\, 1/4,\, 1/4)$. For $y = (1, 0, 0)$,

$$L = -\log \frac{1}{2} = \ln 2, \qquad \nabla_z L = \Big(-\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\Big).$$

The residual sums to 0.

Example 4.2 (Two classes collapse to a sigmoid)**.** For $K = 2$, $p_2 = \sigma(z_2 - z_1)$. Indeed

$$p_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2}} = \frac{1}{e^{z_1 - z_2} + 1} = \sigma(z_2 - z_1).$$

If $y = (0, 1)$, Proposition ?? says $\partial L/\partial z_2 = p_2 - 1 = \sigma(z_2 - z_1) - 1$, which is Proposition ?? applied to the logit difference, with label 1. Binary logistic regression and two-class softmax are the same model written in two coordinate systems.

5 A complete forward pass

Example 5.1 (Both hidden units active)**.** Architecture $2 \to 2 \to 1$, ReLU hidden layer, sigmoid output, binary cross-entropy. One example:

$$x = \begin{pmatrix} 1 \\ 0 \end{pmatrix},\ y = 1,\ W^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix},\ b^{(1)} = \begin{pmatrix} 0 \\ -1/2 \end{pmatrix},\ W^{(2)} = \begin{pmatrix} 1/2 & -1 \end{pmatrix},\ b^{(2)} = 0. \tag{8}$$

Layer 1:

$$z^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -1/2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, \qquad a^{(1)} = \mathrm{ReLU}(z^{(1)}) = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}.$$

Both pre-activations are positive, so $\mathrm{ReLU}'(z^{(1)}) = (1, 1)^\top$. Layer 2:

$$z^{(2)} = \frac{1}{2} \cdot 1 + (-1) \cdot \frac{1}{2} + 0 = 0, \qquad \hat{y} = \sigma(0) = \frac{1}{2}, \qquad L = \ln 2.$$

The cache is $(x, z^{(1)}, a^{(1)}, z^{(2)})$. ReLU's backward mask is a function of $z^{(1)}$, not of $a^{(1)}$. If a coordinate of $a^{(1)}$ is zero, the pre-activation might have been negative or exactly zero; the activation value does not say which, and the two cases are exactly the boundary of the subgradient. Store z.

6 Computation graphs

The multivariate chain rule, for $L = L(r)$ and $r = r(u, y)$ and $u = u(w, x)$, is ordinary multiplication along a path. If two paths reach the same variable, their contributions add.

Example 6.1 (Scalar graph)**.** Let $w = 2$, $x = 3$, $y = 4$, $u = wx$, $r = u - y$, and $L = \frac{1}{2}r^2$.

$$u = 6, \qquad r = 2, \qquad L = 2.$$

Local derivatives: $\partial u/\partial w = x = 3$, $\partial u/\partial x = w = 2$, $\partial r/\partial u = 1$, $\partial L/\partial r = r = 2$. Therefore

$$\frac{\partial L}{\partial w} = 2 \cdot 1 \cdot 3 = 6, \qquad \frac{\partial L}{\partial x} = 2 \cdot 1 \cdot 2 = 4.$$

The closed form $L = \frac{1}{2}(wx - y)^2$ gives the same $\partial L/\partial w = (wx - y)x = 6$. Backpropagation is this arithmetic, organized from the loss backward so each intermediate derivative is computed once.

Proposition 6.1 (One dense layer)**.** *Let $z = Wx + b$ with $W \in \mathbb{R}^{m \times d}$, and let $\delta = \nabla_z L$ be known. Then*

$$\nabla_W L = \delta\, x^\top, \qquad \nabla_b L = \delta, \qquad \nabla_x L = W^\top \delta. \tag{9}$$

Proof. The coordinate form $z_i = \sum_{j=1}^{d} W_{ij} x_j + b_i$ gives

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial z_i} x_j = \delta_i x_j, \qquad \frac{\partial L}{\partial b_i} = \delta_i, \qquad \frac{\partial L}{\partial x_j} = \sum_{i=1}^{m} W_{ij} \delta_i,$$

which are the three displayed products. □

Equation (??) is the entire fully connected backward step. A nonlinearity between $z^{(\ell)}$ and $a^{(\ell)}$ inserts one Hadamard product with $f'(z^{(\ell)})$ before (??) is applied.

7 Backpropagation

Proposition 7.1 (Two-layer binary MLP)**.** *Consider $a^{(1)} = f(z^{(1)})$, $z^{(1)} = W^{(1)}x + b^{(1)}$, $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$, $\hat{y} = \sigma(z^{(2)})$, and binary cross-entropy. Then*

$$\delta^{(2)} = \hat{y} - y, \tag{10}$$

$$\nabla_{W^{(2)}} L = \delta^{(2)} (a^{(1)})^\top, \qquad \nabla_{b^{(2)}} L = \delta^{(2)}, \tag{11}$$

$$\delta^{(1)} = \left((W^{(2)})^\top \delta^{(2)} \right) \odot f'(z^{(1)}), \tag{12}$$

$$\nabla_{W^{(1)}} L = \delta^{(1)} x^\top, \qquad \nabla_{b^{(1)}} L = \delta^{(1)}. \tag{13}$$

For an L-layer net the hidden step (??) repeats:

$$\delta^{(\ell)} = \big((W^{(\ell+1)})^\top \delta^{(\ell+1)}\big) \odot f'(z^{(\ell)}).$$

Proof. Equation (??) is Proposition ??. Equations (??) are Proposition ?? at layer 2, whose incoming activation is $a^{(1)}$. The same proposition applied to $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$ yields $\nabla_{a^{(1)}} L = (W^{(2)})^\top \delta^{(2)}$. The elementwise chain rule through $a^{(1)} = f(z^{(1)})$ multiplies by $f'(z^{(1)})$, which is (??). Equations (??) are Proposition ?? at layer 1. □

If the training objective is the mean of n per-example losses, every gradient in Proposition ?? is replaced by the average of the per-example gradients. Equivalently, sum the δ contributions and divide by n. Mixing a mean loss with a sum of gradients rescales the learning rate by n.

Example 7.1 (Backward pass for (??))**.** From Example ??, $\hat{y} = 1/2$, $y = 1$, $a^{(1)} = (1, 1/2)^\top$, and $f' = (1, 1)^\top$.

$$\begin{aligned}
\delta^{(2)} &= -\frac{1}{2}, \\
\nabla_{W^{(2)}} L &= -\frac{1}{2}\begin{pmatrix} 1 & 1/2 \end{pmatrix} = \begin{pmatrix} -1/2 & -1/4 \end{pmatrix}, \qquad \nabla_{b^{(2)}} L = -\frac{1}{2}, \\
\delta^{(1)} &= \begin{pmatrix} 1/2 \\ -1 \end{pmatrix}\Big(-\frac{1}{2}\Big) = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix}, \\
\nabla_{W^{(1)}} L &= \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix}\begin{pmatrix} 1 & 0 \end{pmatrix} = \begin{pmatrix} -1/4 & 0 \\ 1/2 & 0 \end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix}.
\end{aligned}$$

The second column of $\nabla_{W^{(1)}} L$ is zero because $x_2 = 0$, not because of ReLU. Both hidden units are active.

Example 7.2 (One gradient step)**.** Continue Example ?? with $\eta = 1/10$.

$$\begin{aligned}
W^{(2)} &\leftarrow \begin{pmatrix} 1/2 & -1 \end{pmatrix} - \frac{1}{10}\begin{pmatrix} -1/2 & -1/4 \end{pmatrix} = \begin{pmatrix} 11/20 & -39/40 \end{pmatrix}, \\
b^{(2)} &\leftarrow 0 - \frac{1}{10}\Big(-\frac{1}{2}\Big) = \frac{1}{20}, \\
W^{(1)} &\leftarrow \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} - \frac{1}{10}\begin{pmatrix} -1/4 & 0 \\ 1/2 & 0 \end{pmatrix} = \begin{pmatrix} 41/40 & -1 \\ 19/20 & 1 \end{pmatrix}, \\
b^{(1)} &\leftarrow \begin{pmatrix} 0 \\ -1/2 \end{pmatrix} - \frac{1}{10}\begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix} = \begin{pmatrix} 1/40 \\ -11/20 \end{pmatrix}.
\end{aligned}$$

Recompute the forward map on the same $x = (1, 0)^\top$:

$$\begin{aligned}
z^{(1)} &= \begin{pmatrix} 41/40 \\ 19/20 \end{pmatrix} + \begin{pmatrix} 1/40 \\ -11/20 \end{pmatrix} = \begin{pmatrix} 21/20 \\ 2/5 \end{pmatrix}, \\
a^{(1)} &= z^{(1)} \quad \text{(both coordinates stay positive)}, \\
z^{(2)} &= \frac{11}{20}\cdot\frac{21}{20} + \Big(-\frac{39}{40}\Big)\cdot\frac{2}{5} + \frac{1}{20} = \frac{231 - 156 + 20}{400} = \frac{19}{80}.
\end{aligned}$$

Now $\hat{y} = \sigma(19/80) = \sigma(0.2375) \approx 0.559$ and $L = -\log \hat{y} \approx 0.581$, down from $\ln 2 \approx 0.693$. The step moved the prediction toward the label. A full training run repeats this on a batch, for many steps. Choosing η, averaging noise, and adding momentum are Week 2.

8 Batch shapes, and the NumPy layout

Home Practice stores a batch as $X \in \mathbb{R}^{n \times d}$, one example per row, and uses

$$Z^{(1)} = XW^{(1)}_{\text{code}} + b^{(1)},$$

with $W^{(1)}_{\text{code}} \in \mathbb{R}^{d\times h}$. That matrix is the transpose of the board matrix $W^{(1)} \in \mathbb{R}^{h\times d}$. Biases broadcast across the n rows.

For a mean binary loss the code in Home Practice sets

$$dZ^{(2)} = \frac{\hat{Y} - Y}{n} \in \mathbb{R}^{n\times 1}$$

and then

$$\begin{aligned} dW^{(2)}_{\text{code}} &= (A^{(1)})^\top dZ^{(2)}, & db^{(2)} &= \sum_{\text{rows}} dZ^{(2)}, \\ dA^{(1)} &= dZ^{(2)}(W^{(2)}_{\text{code}})^\top, & dZ^{(1)} &= dA^{(1)} \odot \mathbf{1}[Z^{(1)} > 0], \\ dW^{(1)}_{\text{code}} &= X^\top dZ^{(1)}, & db^{(1)} &= \sum_{\text{rows}} dZ^{(1)}. \end{aligned}$$

On Example **??** with $n = 1$, the board row $\nabla_{W^{(2)}} L = (-1/2, -1/4)$ and the code column $dW^{(2)}_{\text{code}} = (-1/2, -1/4)^\top$ are transposes of each other. They describe one linear map.

Autograd check. Copy the NumPy weights into `torch.nn.Parameter`, run the same forward expression, call `loss.backward()`, and compare. A relative error below 10^{-5} on every parameter is the acceptance test in Home Practice. The usual failures are a missing $1/n$, a bias reduced with the wrong axis, or a ReLU mask built from A instead of Z.

9 Problems

Solutions start in Section **??**. Exact answers are expected.

P1. Compute $\sigma(0)$, $\sigma'(0)$, and $\tanh'(0)$. Prove $\sigma'(z) \le 1/4$.

P2. Prove Proposition **??** from the definition, without looking it up mid-proof. Then prove Proposition **??**.

P3. Complete the linear-separability contradiction for XOR in your own lines. Then evaluate the ReLU network of Example **??** at $(1, 1)$ and at $(1, 0)$, showing each pre-activation.

P4. How many parameters does a fully connected network $4 \to 3 \to 2$ have? Write every matrix shape in the board convention. For a batch of 32 rows in the code convention, write the shapes of X, $W^{(1)}_{\text{code}}$, $Z^{(1)}$, and $dW^{(1)}_{\text{code}}$.

P5. Let $z = (0, \ln 3)$ and $y = (0, 1)$. Compute $\text{softmax}(z)$, the cross-entropy, and $\nabla_z L$. Verify $\partial L/\partial z_2 = \sigma(z_2 - z_1) - 1$.

P6. (Main calculation.) Use

$$x = \begin{pmatrix} 1 \\ 1 \end{pmatrix},\ y = 0,\ W^{(1)} = I_2,\ b^{(1)} = \begin{pmatrix} -1/2 \\ -3/2 \end{pmatrix},\ W^{(2)} = \begin{pmatrix} 2 & 2 \end{pmatrix},\ b^{(2)} = -1,$$

with ReLU, a sigmoid output, and binary cross-entropy. Compute the forward pass and every parameter gradient. Identify the dead unit and list the gradient entries it forces to zero.

P7. Using the gradients of Example **??** and $\eta = 1/10$, recompute $W^{(2)}$ and $b^{(2)}$ only (leave layer 1 unchanged). Evaluate the new $z^{(2)}$ with the original $a^{(1)} = (1, 1/2)^\top$, and show that the new $\hat{y}$ is larger than $1/2$.

P8. Explain, in four to six sentences, why the backward pass for ReLU must consult $z^{(1)}$. Then explain what happens to $\nabla_{W^{(1)}} L$ on a given example if every hidden unit is dead on that example.

10 Solutions

P1

$$\sigma(0) = \frac{1}{2}, \qquad \sigma'(0) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}, \qquad \tanh'(0) = 1 - \tanh^2(0) = 1.$$

The bound is the second half of Proposition **??**: $s(1-s) \le 1/4$ for $s \in (0,1)$.

P2

The proofs are Proposition **??** and Proposition **??**. A solution that starts from $\partial L/\partial \hat{y}$ and multiplies by σ' is complete; a solution that only quotes $\hat{y} - y$ is not.

P3

The contradiction is Example **??**. At $(1, 1)$,

$$z^{(1)} = (2,\ 1)^\top, \quad a = (2,\ 1)^\top, \quad \hat{y} = 2 - 2 \cdot 1 = 0.$$

At $(1, 0)$,

$$z^{(1)} = (1,\ 0)^\top, \quad a = (1,\ 0)^\top, \quad \hat{y} = 1.$$

P4

Board shapes: $W^{(1)} \in \mathbb{R}^{3\times 4}$, $b^{(1)} \in \mathbb{R}^3$, $W^{(2)} \in \mathbb{R}^{2\times 3}$, $b^{(2)} \in \mathbb{R}^2$. Count

$$(3 \cdot 4 + 3) + (2 \cdot 3 + 2) = 15 + 8 = 23.$$

Code shapes for $n = 32$: X is 32×4, $W^{(1)}_{\text{code}}$ is 4×3, $Z^{(1)}$ is 32×3, and $dW^{(1)}_{\text{code}}$ has the same shape as $W^{(1)}_{\text{code}}$, namely 4×3.

P5

$e^z = (1, 3)$ and the normalizer is 4, so $p = (1/4,\ 3/4)$. The true class is class 2, hence

$$L = -\log \frac{3}{4} = \log \frac{4}{3}, \qquad \nabla_z L = \Big(\frac{1}{4}, -\frac{1}{4}\Big).$$

The logit difference is $z_2 - z_1 = \ln 3$ and

$$\sigma(\ln 3) = \frac{1}{1 + 1/3} = \frac{3}{4}, \qquad \sigma(\ln 3) - 1 = -\frac{1}{4} = \frac{\partial L}{\partial z_2}.$$

P6

Forward:

$$z^{(1)} = \begin{pmatrix} 1 - 1/2 \\ 1 - 3/2 \end{pmatrix} = \begin{pmatrix} 1/2 \\ -1 \end{pmatrix}, \qquad a^{(1)} = \begin{pmatrix} 1/2 \\ 0 \end{pmatrix}.$$

$$z^{(2)} = 2 \cdot \frac{1}{2} + 2 \cdot 0 - 1 = 0, \qquad \hat{y} = \frac{1}{2}, \qquad L = -\log \frac{1}{2} = \ln 2$$

because $y = 0$. Backward:

$$\delta^{(2)} = \frac{1}{2} - 0 = \frac{1}{2},$$

$$\nabla_{W^{(2)}} L = \frac{1}{2} \begin{pmatrix} 1/2 & 0 \end{pmatrix} = \begin{pmatrix} 1/4 & 0 \end{pmatrix}, \qquad \nabla_{b^{(2)}} L = \frac{1}{2},$$

$$\delta^{(1)} = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \cdot \frac{1}{2} \odot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

$$\nabla_{W^{(1)}} L = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

Unit 2 is dead because $z^{(1)}_2 = -1 < 0$. That zero in the ReLU mask forces the second row of $\nabla_{W^{(1)}} L$ and the second entry of $\nabla_{b^{(1)}} L$ to vanish. The outgoing weight is also zero in the gradient, for a second reason: $a^{(1)}_2 = 0$, so the second column of $\nabla_{W^{(2)}} L = \delta^{(2)} (a^{(1)})^\top$ vanishes even before one thinks about the mask. On this example the dead unit is frozen: incoming weights, bias, and outgoing weight do not move.

P7

Leave $a^{(1)} = (1,\, 1/2)^\top$ as in Example **??**. From Example **??**,

$$W^{(2)} = \begin{pmatrix} 11/20 & -39/40 \end{pmatrix}, \qquad b^{(2)} = 1/20.$$

$$z^{(2)} = \frac{11}{20} \cdot 1 + \Big(-\frac{39}{40} \Big) \cdot \frac{1}{2} + \frac{1}{20} = \frac{22}{40} - \frac{39}{80} + \frac{4}{80} = \frac{44 - 39 + 4}{80} = \frac{9}{80} > 0.$$

Thus $\hat{y} = \sigma(9/80) > \sigma(0) = 1/2$. Updating only the output layer already moves the prediction toward $y = 1$. Updating layer 1 as well, which Example **??** does, moves it further, to $\sigma(19/80)$.

P8

ReLU's derivative at a coordinate is 1 when the pre-activation is positive and 0 when it is negative. The activation stores only the positive part, so the value 0 is compatible with a negative pre-activation and with a zero pre-activation. The mask has to be computed from $z^{(1)}$. If every hidden unit is dead on an example, then $f'(z^{(1)}) = 0$, hence $\delta^{(1)} = 0$, hence $\nabla_{W^{(1)}} L = \delta^{(1)} x^\top = 0$ and $\nabla_{b^{(1)}} L = 0$ on that example. The example does not teach the first layer anything. If this happens for every example in the batch, the first-layer update is the zero matrix.
